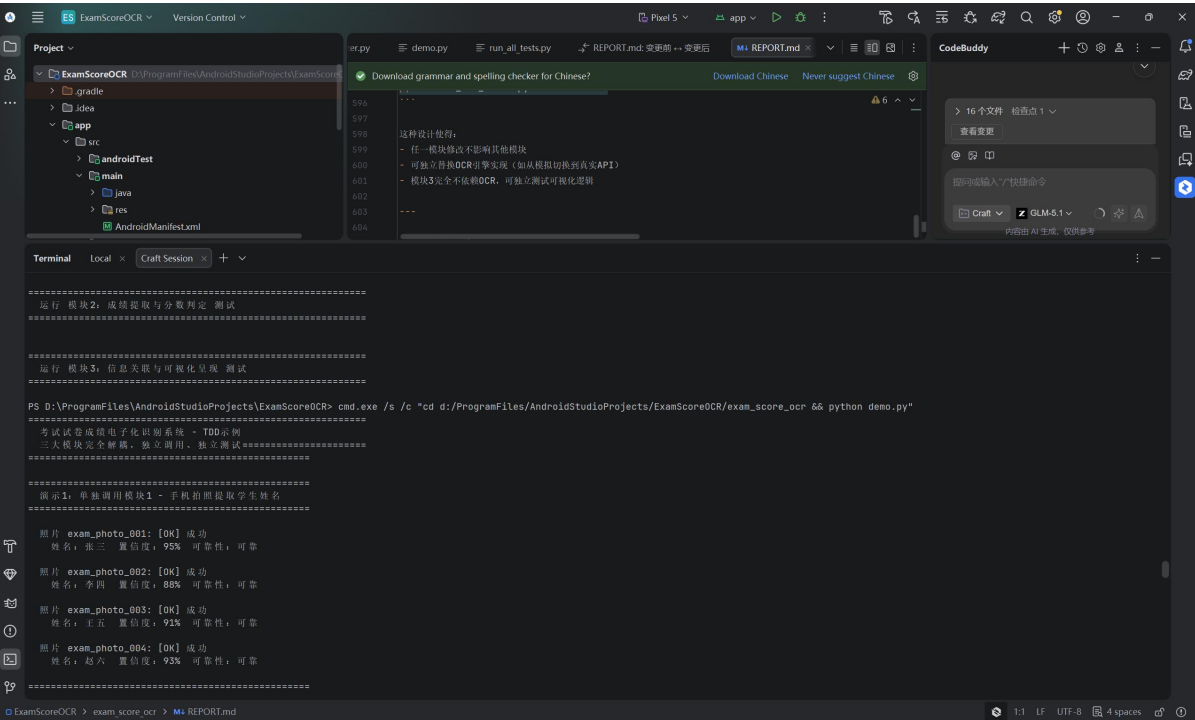


基于 TDD 测试驱动开发的考试试卷成绩电子化识别系统作业报告



一、项目概述

1.1 业务背景

纸质期末试卷在考后需要人工录入成绩，存在效率低、易出错等问题。本系统通过模拟手机拍照 OCR 识别能力，实现试卷关键信息的自动提取与电子化：

- 试卷左上角：学生姓名
- 试卷首页顶部：卷面总分成绩

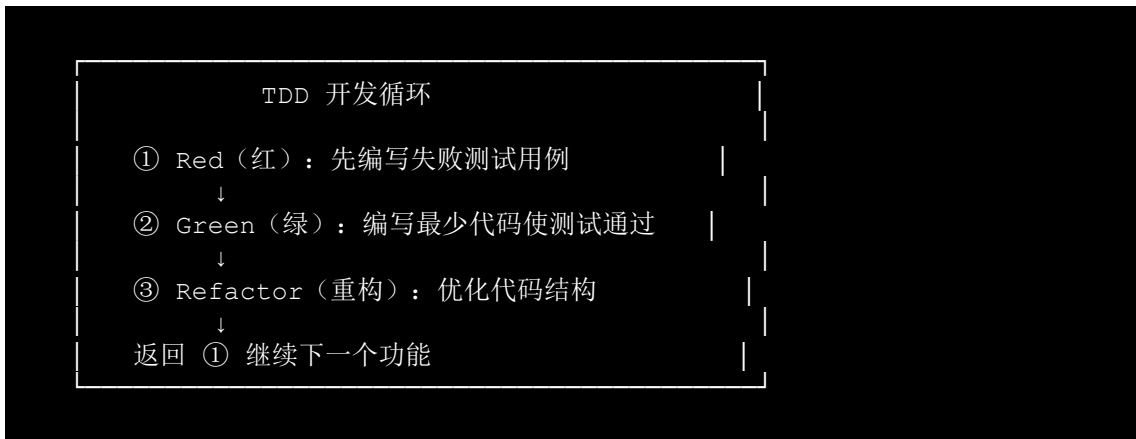
1.2 核心目标

- 严格遵循 TDD（测试驱动开发）流程：先写测试 → 再写实现 → 重构优化
- 三大功能模块 完全解耦，可独立调用、独立测试
- 模拟手机拍照 OCR 识别能力，本地可运行测试

二、TDD 开发方法论

2.1 TDD 核心循环

本项目严格遵循 TDD 的 **Red-Green-Refactor** 三步循环：

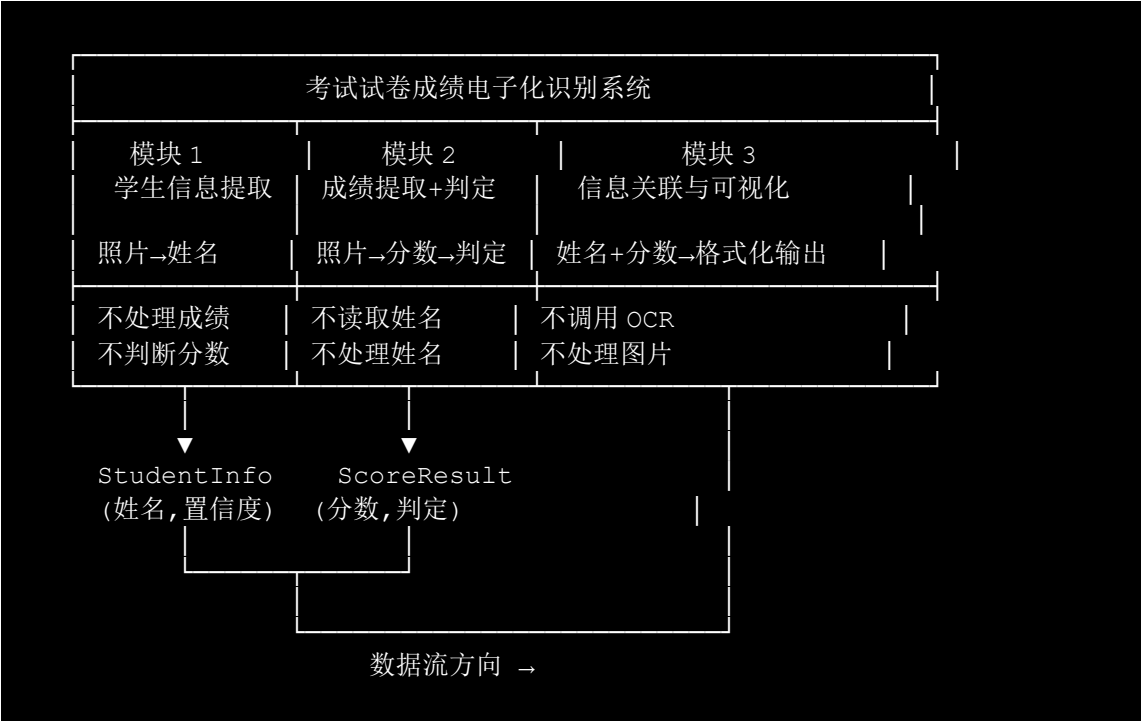


2.2 本项目 TDD 实践流程

阶段	模块 1	模块 2	模块 3
Red (先写测试)	编写 10 个测试用例	编写 15 个测试用例	编写 10 个测试用例
Green (再写实现)	实现 StudentExtractor	实现 ScoreExtractor	实现 ResultVisualizer
Refactor (重构)	抽取 3 个私有方法	抽取 2 个私有方法	拆分 3 种格式输出方法
测试结果	10/10 通过	15/15 通过	10/10 通过

三、系统架构设计

3.1 模块划分



3.2 项目文件结构

```
exam_score_ocr/  
├── __init__.py  
├── ocr_simulator/  
│   ├── __init__.py  
│   └── ocr_engine.py  
├── module_student/  
│   ├── __init__.py  
│   ├── test_student_extractor.py  
│   └── student_extractor.py  
├── module_score/  
│   ├── __init__.py  
│   ├── test_score_extractor.py  
│   └── score_extractor.py  
├── module_visualization/  
│   ├── __init__.py  
│   ├── test_result_visualizer.py  
│   └── result_visualizer.py  
├── demo.py  
├── run_all_tests.py  
└── README.md
```

项目包初始化
模拟 OCR 工具模块

模拟 OCR 识别引擎
模块 1：学生信息提取

【TDD 第一步】先写的单元测试
【TDD 第二步】后写的业务实现
模块 2：成绩提取与分数判定

【TDD 第一步】先写的单元测试
【TDD 第二步】后写的业务实现
模块 3：信息关联与可视化呈现

【TDD 第一步】先写的单元测试
【TDD 第二步】后写的业务实现
完整调用演示
测试运行脚本
使用说明

3.3 模块职责与解耦设计

模块	类名	职责	输入	输出	明确不做什么
模块 1	StudentExtractor	手机拍照提取学生姓名	照片 ID	StudentInfo	不处理成绩、不判断分数
模块 2	ScoreExtractor	手机拍照提取成绩+判定	照片 ID	ScoreResult	不读取学生姓名
模块 3	ResultVisualizer	信息关联与可视化呈现	StudentInfo + ScoreResult	VisualizationOutput	不调用 OCR、不处理图片

解耦关键：三大模块代码文件/类完全分离，互不依赖，仅通过 StudentInfo 和 ScoreResult 数据对象传递数据。模块 3 不直接调用 OCR 引擎，只接收前两个模块的输出结果。

四、分模块 TDD 开发详解

4.1 模拟 OCR 引擎（基础设施）

在开始 TDD 之前，先搭建模拟 OCR 基础设施，使测试可以脱离真实第三方 API 运行。

```
# ocr_simulator/ocr_engine.py 核心设计
class OcrEngine:
    """模拟 OCR 引擎 - 内置预设数据，模拟手机拍照识别"""

    DEFAULT MOCK_DATA = {
        "exam_photo_001": [ # 张三 85 分
            OcrRegion(text="姓名: 张三", confidence=0.95, x=10,
y=10, ...),
            OcrRegion(text="卷面总分: 85", confidence=0.92, x=200,
y=5, ...),
        ],
        # ... 共 11 组预设照片数据，覆盖各种场景
    }
```

```
def recognize(self, photo_id: str) -> OcrResult:
    """根据图片 ID 返回预设识别结果"""
    ...
```

预设数据覆盖场景：正常及格/不及格/60 分边界/0 分/只有姓名/只有成绩/空白试卷/模糊照片/负数分数/超大分数/非数字成绩，共 11 组。

4.2 模块 1：学生信息提取 —— TDD 全过程

第一步：Red（先写测试）

在 `test_student_extractor.py` 中，先于业务代码编写 10 个测试用例：

```
class TestStudentExtractor(unittest.TestCase):

    def setUp(self):
        self.ocr_engine = OcrEngine()
        self.extractor = StudentExtractor(self.ocr_engine)

    # 测试 1：正常识别
    def test_extract_normal_student(self):
        result = self.extractor.extract("exam_photo_001")
        self.assertTrue(result.success)
        self.assertEqual(result.name, "张三")
        self.assertGreater(result.confidence, 0.8)

    # 测试 2：空照片 ID
    def test_extract_empty_photo_id(self):
        result = self.extractor.extract("")
        self.assertFalse(result.success)
        self.assertIn("无效", result.error_message)

    # 测试 3：不存在的照片 ID
    def test_extract_nonexistent_photo(self):
        result = self.extractor.extract("nonexistent_photo")
        self.assertFalse(result.success)
        self.assertIn("未找到", result.error_message)

    # 测试 4：空白试卷
    def test_extract_blank_exam(self):
        result = self.extractor.extract("exam_photo_blank")
        self.assertFalse(result.success)
```

```

        self.assertIn("未检测到姓名", result.error_message)

# 测试 5: 只有成绩无姓名
def test_extract_only_score_no_name(self):
    result = self.extractor.extract("exam_photo_only_score")
    self.assertFalse(result.success)
    self.assertIn("未检测到姓名", result.error_message)

# 测试 6: OCR 低置信度
def test_extract_low_confidence(self):
    result = self.extractor.extract("exam_photo_blurry")
    if result.success:
        self.assertFalse(result.reliable)

# 测试 7-10: 自定义数据、无前缀姓名等...

```

此时运行测试，全部失败（因为 `StudentExtractor` 尚未实现），这是 TDD 的 **Red** 阶段。

第二步：Green（编写实现使测试通过）

```

# module_student/student_extractor.py
class StudentExtractor:
    """模块 1: 学生信息提取器 - 仅提取姓名，不处理成绩"""

    def __init__(self, ocr_engine: OcrEngine):
        self.ocr_engine = ocr_engine # 依赖注入，解耦 OCR 引擎

    def extract(self, photo_id: str) -> StudentInfo:
        # 1. 参数校验
        if not photo_id or not isinstance(photo_id, str):
            return StudentInfo(success=False, error_message="图片 ID
无效")

        # 2. 调用 OCR 识别
        ocr_result = self.ocr_engine.recognize(photo_id)
        # 3. 处理 OCR 失败/无结果
        if not ocr_result.success:
            return StudentInfo(success=False,
error_message=ocr_result.error_message)
        # 4. 查找姓名区域
        name_region = self._find_name_region(ocr_result.regions)
        # 5. 解析姓名文本
        name = self._parse_name(name_region.text)
        # 6. 检查置信度
        reliable = name_region.confidence >= CONFIDENCE_THRESHOLD
        return StudentInfo(success=True, name=name, confidence=...,
reliable=reliable)

```

运行测试，10 个用例全部通过，进入 **Green** 阶段。

第三步：Refactor（重构优化）

将核心逻辑抽取为 3 个私有方法：

- `_find_name_region()`：按关键字查找姓名区域，排除成绩关键字
- `_find_top_left_region()`：查找左上角区域作为 fallback
- `_parse_name()`：从原始文本解析姓名，支持多种前缀格式

TDD 过程中发现的 Bug： 初始实现中 `_find_top_left_region()` 没有排除成绩关键字区域，导致"只有成绩无姓名"的照片错误地把成绩文本当成了姓名。正是测试用例 `test_extract_only_score_no_name` 捕获了这个 Bug，体现了 TDD 的价值。

4.3 模块 2：成绩提取与分数判定 —— TDD 全过程

第一步：Red（先写测试）

编写 15 个测试用例，覆盖：

编号	测试场景	预期结果
1	85 分	judgment="及格"
2	56 分	judgment="不及格"
3	60 分（边界）	judgment="及格"
4	0 分	judgment="不及格"
5-6	空 ID/None	返回"图片 ID 无效"
7	不存在的 ID	返回"图片未找到"
8	空白试卷	返回"未检测到成绩"
9	只有姓名无成绩	返回"未检测到成绩"
10	低置信度	reliable=False

编号	测试场景	预期结果
11	负数分数(-5)	返回"分数异常"
12	超大分数(999)	返回"分数异常"
13	非数字(ABC)	返回"分数解析失败"
14-15	自定义 75 分/100 分	正常判定

```
# 关键边界测试用例示例
def test_extract_boundary_60(self):
    """恰好 60 分：应为及格（含边界）"""
    result = self.extractor.extract("exam_photo_003")
    self.assertTrue(result.success)
    self.assertEqual(result.score, 60)
    self.assertEqual(result.judgment, "及格")

def test_extract_negative_score(self):
    """负数分数：应返回分数异常"""
    result = self.extractor.extract("exam_photo_negative_score")
    self.assertFalse(result.success)
    self.assertIn("异常", result.error_message)
```

第二步：Green（编写实现）

```
class ScoreExtractor:
    """模块 2：成绩提取与分数判定器 - 仅处理分数，不读取姓名"""

    def extract(self, photo_id: str) -> ScoreResult:
        # 1-4. 参数校验、OCR 识别、失败处理（同模块 1）
        # 5. 查找成绩区域
        score_region = self._find_score_region(ocr_result.regions)
        # 6. 解析分数数字
        score = self._parse_score(score_region.text)
        # 7. 校验分数范围 [0, 100]
        if score < 0 or score > 100:
            return ScoreResult(success=False, error_message="分数异常")

        # 8. 执行判定
        judgment = "及格" if score >= 60 else "不及格"
        # 9. 检查置信度
        ...
```

第三步：Refactor（重构优化）

- 抽取 `_find_score_region()` 和 `_parse_score()` 私有方法
 - 统一定义常量： `PASS_THRESHOLD=60`, `MIN_SCORE=0`, `MAX_SCORE=100`
 - 成绩前缀列表 `SCORE_PREFIXES` 支持多种格式匹配
-

4.4 模块 3：信息关联与可视化呈现 —— TDD 全过程

第一步：Red（先写测试）

编写 10 个测试用例：

编号	测试场景	预期结果
1	完整数据	<code>complete=True</code>
2	姓名缺失	<code>name="（姓名缺失）"</code>
3	分数缺失	<code>score_display="（分数缺失）"</code>
4	都缺失	<code>warnings</code> 有 2 条
5	及格结果	<code>judgment="及格"</code>
6	不及格结果	<code>judgment="不及格"</code>
7	不可靠识别	<code>warnings</code> 非空
8	纯文本格式	输出包含姓名和分数
9	表格格式	输出包含姓名和分数
10	JSON 格式	可正确解析

```
def test_visualize_missing_name(self):
    """模块 1 返回失败时，姓名标记为缺失"""
    student_info = StudentInfo(success=False, name="",
error_message="未检测到姓名")
    score_result = ScoreResult(success=True, score=72, judgment="及格", ...)
    output = self.visualizer.visualize(student_info, score_result)
    self.assertFalse(output.complete)
    self.assertEqual(output.name, "（姓名缺失）")
```

```

        self.assertEqual(output.score, 72)

    def test_format_json(self):
        """JSON 格式输出应可正确解析"""
        output = self.visualizer.visualize(student_info, score_result)
        json_str = self.visualizer.format_json(output)
        data = json.loads(json_str)
        self.assertEqual(data["name"], "王五")
        self.assertEqual(data["score"], 60)
        self.assertEqual(data["judgment"], "及格")

```

第二步：Green（编写实现）

```

class ResultVisualizer:
    """模块 3：信息关联与可视化 - 不调用 OCR、不处理图片"""

    def visualize(self, student_info, score_result) ->
VisualizationOutput:
        # 整合两个模块的输出，处理缺失情况
        name = student_info.name if student_info.success else "（姓名缺失）"
        score_display = str(score_result.score) if
score_result.success else "（分数缺失）"
        # 收集警告信息
        warnings = []
        if not student_info.success: warnings.append(...)
        if not score_result.success: warnings.append(...)
        return VisualizationOutput(complete=..., name=name, ...)

    def format_text(self, output): ...    # 纯文本格式
    def format_table(self, output): ...  # 表格格式
    def format_json(self, output): ...   # JSON 格式

```

五、测试结果

5.1 全部测试通过

```

$ python run_all_tests.py -v

模块 1：学生信息提取 - Ran 10 tests - OK
模块 2：成绩提取与分数判定 - Ran 15 tests - OK
模块 3：信息关联与可视化呈现 - Ran 10 tests - OK

```

总计：35 个测试用例，全部通过

5.2 完整测试用例清单

编号	模块	测试方法	测试场景	结果
1	模块 1	test_extract_normal_student	正常识别张三	PASS
2	模块 1	test_extract_another_normal_student	正常识别李四	PASS
3	模块 1	test_extract_empty_photo_id	空照片 ID	PASS
4	模块 1	test_extract_none_photo_id	None 照片 ID	PASS
5	模块 1	test_extract_nonexistent_photo	不存在的照片 ID	PASS
6	模块 1	test_extract_blank_exam	空白试卷	PASS
7	模块 1	test_extract_only_score_no_name	只有成绩无姓名	PASS
8	模块 1	test_extract_low_confidence	OCR 低置信度	PASS
9	模块 1	test_extract_with_custom_data	自定义数据注入	PASS
10	模块 1	test_extract_name_without_prefix	无前缀姓名格式	PASS
11	模块 2	test_extract_pass_score	85 分及格	PASS
12	模块 2	test_extract_fail_score	56 分不及格	PASS
13	模块 2	test_extract_boundary_60	60 分边界及格	PASS
14	模块 2	test_extract_zero_score	0 分不及格	PASS
15	模块 2	test_extract_empty_photo_id	空照片 ID	PASS
16	模块 2	test_extract_none_photo_id	None 照片 ID	PASS
17	模块 2	test_extract_nonexistent_photo	不存在的照片 ID	PASS
18	模块 2	test_extract_blank_exam	空白试卷	PASS
19	模块 2	test_extract_only_name_no_score	只有姓名无成绩	PASS

编号	模块	测试方法	测试场景	结果
		ore		
20	模块 2	test_extract_low_confidence	低置信度	PASS
21	模块 2	test_extract_negative_score	负数分数(-5)	PASS
22	模块 2	test_extract_huge_score	超大分数(999)	PASS
23	模块 2	test_extract_invalid_score	非数字(ABC)	PASS
24	模块 2	test_extract_custom_pass	自定义 75 分	PASS
25	模块 2	test_extract_full_score	100 分满分	PASS
26	模块 3	test_visualize_complete_data	完整数据	PASS
27	模块 3	test_visualize_missing_name	姓名缺失	PASS
28	模块 3	test_visualize_missing_score	分数缺失	PASS
29	模块 3	test_visualize_both_missing	都缺失	PASS
30	模块 3	test_visualize_pass_result	及格结果	PASS
31	模块 3	test_visualize_fail_result	不及格结果	PASS
32	模块 3	test_visualize_unreliable	不可靠识别	PASS
33	模块 3	test_format_text	纯文本格式	PASS
34	模块 3	test_format_table	表格格式	PASS
35	模块 3	test_format_json	JSON 格式	PASS

六、运行演示

6.1 单独提取姓名（模块 1 独立调用）

```
照片 exam_photo_001: [OK] 成功
  姓名: 张三   置信度: 95%   可靠性: 可靠
照片 exam_photo_002: [OK] 成功
  姓名: 李四   置信度: 88%   可靠性: 可靠
```

6.2 单独识别判分（模块 2 独立调用）

```
照片 exam_photo_001: [OK] 成功
  分数: 85  判定: 及格  置信度: 92%  可靠性: 可靠
照片 exam_photo_002: [OK] 成功
  分数: 56  判定: 不及格  置信度: 85%  可靠性: 可靠
```

6.3 完整业务流程（模块 1→模块 2→模块 3 联调）

```
[拍照] 处理: 张三的试卷 (照片 ID: exam_photo_001)
=====
      考试试卷成绩识别结果
=====
  学生姓名: 张三
  卷面分数: 85
  判定结果: 及格
=====

[拍照] 处理: 李四的试卷 (照片 ID: exam_photo_002)
=====
      考试试卷成绩识别结果
=====
  学生姓名: 李四
  卷面分数: 56
  判定结果: 不及格
=====
```

6.4 异常场景处理

```
[拍照] 场景: 负数分数 (照片 ID: 'exam_photo_negative_score')
=====
      考试试卷成绩识别结果
=====
  学生姓名: 吴九
  卷面分数: (分数缺失)
  判定结果:
=====

[!] 警告信息:
  - 成绩识别异常: 分数异常: -5 不在有效范围[0, 100]内
=====

[拍照] 场景: 模糊照片 (低置信度)
=====
      考试试卷成绩识别结果
```

```
=====
学生姓名: 周八
卷面分数: 91
判定结果: 及格
=====
```

```
[!] 警告信息:
- 姓名识别不可靠 (置信度: 35%)
- 成绩识别不可靠 (置信度: 30%)
=====
```

6.5 多格式输出

纯文本格式:

```
=====
          考试试卷成绩识别结果
=====
学生姓名: 张三
卷面分数: 85
判定结果: 及格
=====
```

表格格式:

```
+-----+-----+
| 项目           | 内容           |
+-----+-----+
| 学生姓名       | 张三           |
| 卷面分数       | 85             |
| 判定结果       | 及格           |
+-----+-----+
```

JSON 格式:

```
{
  "complete": true,
  "name": "张三",
  "score": 85,
  "score_display": "85",
  "judgment": "及格",
  "warnings": []
}
```

七、边界测试用例汇总

编号	边界场景	所属模块	输入	预期输出	实际结果
B01	正常及格分数(85)	模块 2	exam_photo_001	score=85, judgment="及格"	PASS
B02	正常不及格分数(56)	模块 2	exam_photo_002	score=56, judgment="不及格"	PASS
B03	恰好 60 分(边界值)	模块 2	exam_photo_003	score=60, judgment="及格"	PASS
B04	0 分	模块 2	exam_photo_004	score=0, judgment="不及格"	PASS
B05	100 分满分	模块 2	自定义数据	score=100, judgment="及格"	PASS
B06	空照片 ID	模块 1/2	""	返回"图片 ID 无效"	PASS
B07	None 照片 ID	模块 1/2	None	返回"图片 ID 无效"	PASS
B08	不存在的照片 ID	模块 1/2	"nonexistent"	返回"图片未找到"	PASS
B09	空白试卷(无文字)	模块 1/2	exam_photo_blank	返回"未检测到"	PASS
B10	只有姓名无成绩	模块 2	exam_photo_only_name	返回"未检测到成绩"	PASS
B11	只有成绩无姓名	模块 1	exam_photo_only_score	返回"未检测到姓名"	PASS
B12	OCR 低置信度(<0.5)	模块 1/2	exam_photo_blurry	reliable=False	PASS
B13	负数分数(-5)	模块 2	exam_photo_neg	返回"分数异常"	PASS

编号	边界场景	所属模块	输入	预期输出	实际结果
			ative_score		
B14	超大分数(999)	模块 2	exam_photo_huge_score	返回"分数异常"	PASS
B15	非数字成绩(ABC)	模块 2	exam_photo_invalid_score	返回"分数解析失败"	PASS
B16	无"姓名: "前缀	模块 1	自定义数据	仍能提取姓名	PASS
B17	姓名和分数都缺失	模块 3	两个失败结果	warnings 有 2 条	PASS
B18	不可靠识别(低置信度)	模块 3	unreliable=True	warnings 非空	PASS

八、TDD 实践总结与心得

8.1 TDD 带来的收益

- 测试先行暴露设计缺陷：**模块 1 开发中，`test_extract_only_score_no_name` 测试用例提前发现了"成绩区域被误识别为姓名"的 Bug，在业务代码编写前就明确了需求边界。
- 解耦设计自然形成：**因为要先写测试，必须先定义好输入输出的数据结构（`StudentInfo`、`ScoreResult`），这促使模块间通过数据对象传递，而非直接调用，实现了天然的低耦合。
- 重构有安全网：**`Refactor` 阶段可以放心优化代码结构，因为所有测试用例构成了安全网，任何回归错误都能被立即捕获。
- 边界用例提前覆盖：**负数分数、超大分数、非数字成绩等边界场景在测试阶段就已设计，确保实现代码对这些异常都有处理，而非事后补救。

8.2 AI 辅助编程在 TDD 中的应用

在本项目中，AI 辅助编程发挥了以下作用：

- **测试用例生成**: AI 根据业务需求快速生成全面的测试用例, 覆盖正常/异常/边界场景, 避免遗漏
- **代码骨架生成**: AI 根据测试用例生成业务实现代码骨架, 开发者聚焦于核心逻辑
- **Bug 定位与修复**: 测试失败时, AI 协助分析错误原因并提出修复方案 (如 `_find_top_left_region` 排除成绩关键字的修复)
- **重构建议**: AI 识别代码重复, 建议抽取公共方法

8.3 模块解耦的价值

三大模块完全独立, 各自可单独运行和测试:

```
# 单独运行模块 1 测试
python run_all_tests.py --module 1 -v

# 单独运行模块 2 测试
python run_all_tests.py --module 2 -v

# 单独运行模块 3 测试
python run_all_tests.py --module 3 -v
```

这种设计使得:

- 任一模块修改不影响其他模块
- 可独立替换 OCR 引擎实现 (如从模拟切换到真实 API)
- 模块 3 完全不依赖 OCR, 可独立测试可视化逻辑

九、运行方式

```
cd exam_score_ocr

# 运行全部 35 个单元测试
python run_all_tests.py -v

# 运行指定模块测试
python run_all_tests.py --module 1 -v
python run_all_tests.py --module 2 -v
python run_all_tests.py --module 3 -v
```

```
# 运行完整业务演示
python demo.py
```

十、结论

本项目严格遵循 TDD 测试驱动开发流程，成功构建了考试试卷成绩电子化识别系统。三大核心模块（学生信息提取、成绩提取与判定、信息关联与可视化）完全解耦，通过 35 个单元测试覆盖正常场景与 18 种边界场景，全部测试通过。模拟 OCR 引擎使系统可在本地独立运行测试，验证了 TDD 方法论在保证代码质量和设计合理性方面的有效性。